XRComm FCS 8T8R Platform

Quick Start & User Guide

XRComm Inc.

June 2026

XRComm FCS 8T8R Platform

Quick Start & User Guide

Platform	XRComm FCS 8T8R Platform
Driver Version	v1.4.3
Variant	8T8R
Audience	End users and system administrators
Classification	Proprietary

Contents

1
Terminology

The following terms are used throughout this document.

Term	Meaning
FlexCompute Server	The physical host machine running the XRComm platform driver and gRPC control service. All platform binaries execute here.
Client Machine	Any machine (including the FlexCompute Server itself) from which you run the Python gRPC control client to operate the platform.
Platform Driver	The `xrcomm-server` binary. Receives the high-speed RF stream from the NIC, manages shared memory, records to NVMe, and dispatches data to customer applications.
Control Service	The `xrcomm-grpc-ctrl` binary. Provides a gRPC interface (TCP port 50051) for remotely controlling the Platform Driver.
Customer Application	Your own software that connects to the Platform Driver via shared memory to consume IQ samples or full packets.
IQ Delivery	Mode in which interleaved int16 I/Q sample batches are dispatched to connected applications via zero-copy shared memory.
Full-Packet Delivery	Mode in which raw network packets are forwarded to a single application using DPDK secondary process attachment. Requires `sudo`.
NVMe Recording	Mode in which the received RF stream is written directly to the NVMe drive via SPDK for later offline extraction.
8T8R Playback Engine	The `xrcomm_playback` binary and `playback_grpc_server.py`. Streams pre-recorded IQ waveforms from files to the network interface at line rate with per-channel configurations.

2

System Topology

The platform operates as a set of background services on the FlexCompute Server. Remote or local clients can control the platform over the network using a Python gRPC client.

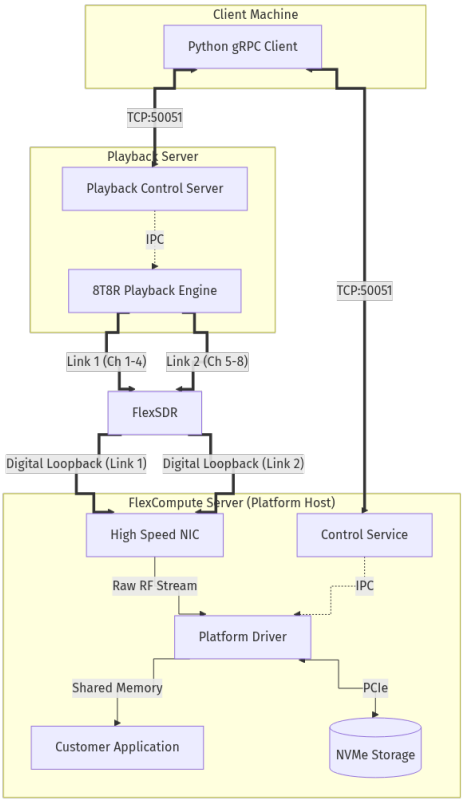

Figure 1: XRComm FCS 8T8R Platform Topology

Data flow:

1. The **High Speed NIC** receives the raw RF stream and delivers it to the **Platform Driver** via DPDK.
2. The **Platform Driver** dispatches IQ samples to connected **Customer Applications** via zero-copy shared memory.
3. Simultaneously, the Platform Driver can record the stream to the **NVMe Storage** via SPDK.
4. The **Control Service** communicates with the Platform Driver over IPC, allowing remote management.
5. The **Python gRPC Client** (on the **Client Machine** or on the FlexCompute Server itself) connects to the Control Service over TCP port 50051.

3

Package Contents

After extracting the tarball, you will find the following directory structure:

```
XRComm_FCS_platform/
  install.sh                    # Automated installer
  uninstall.sh                  # Clean uninstaller
  LICENSE                       # Software license agreement
  VERSION                       # Release version identifier
  CHANGELOG.md                  # Release notes
  8T8R_Quickstart.pdf           # This document
```



```
XRComm_FCS_8T8R_platform/
  config/
    read_config.ini            # 8T8R per-channel auto-read config
  XRComm_platform_drivers/
    bin/                       # Precompiled platform driver binary
    include/                   # Public C headers for app development
    service/                   # Systemd unit file templates
  XRComm_platform_gRPC/
    bin/                       # Precompiled gRPC control service binary
    client/                    # Python gRPC client scripts
    proto/                     # Protobuf service definition (.proto)
    service/                   # Systemd unit file templates

XRComm_8T8R_hello_world/
  hello_world.c                # IQ delivery application (no sudo)
  CMakeLists.txt               # Build configuration
  bin/                         # Output directory for compiled applications

XRComm_8T8R_playback/
  bin/                         # Precompiled playback binaries and server
    xrcomm_playback            # Playback engine binary
    playback_grpc_server.py    # Playback gRPC server
    pipeline_control_pb2.py    # Shipped precompiled server stub
    pipeline_control_pb2_grpc.py # Shipped precompiled server stub
  inputs/                      # Playback config and waveform files
    config.ini                 # Playback configuration
    playback_waveforms/        # Waveform files
  grpc/
    client/                    # Python playback client and shipped stubs
    proto/                     # Playback protobuf definition (.proto)

XRComm_NVMe_loader/
  bin/                         # Precompiled NVMe reader binary
  config/                      # Optional offline reader defaults
  scripts/                     # Optional offline extraction (read_nvme.sh)
```

4 Installation

4.1 Extract the Package

Transfer the tarball to your FlexCompute Server and extract it:

```
tar -xzvf XRComm_FCS_platform.tar.gz
cd XRComm_FCS_platform
```

4.2

Run the Installer

```
sudo ./install.sh
```

When prompted, accept the license agreement and select **Option 2** for the 8T8R Platform.

The installer performs the following actions automatically:

Step	What it does
Dependency installation	Installs `python3-grpcio`, `python3-protobuf`, and `python3-grpc-tools` via `apt`.
Systemd service creation	Creates and enables `xrcomm-server.service` and `xrcomm-grpc-ctrl.service`.
Header file mapping	Symlinks the public C headers into `/usr/include/xrcomm-8T8R/`.
Proto compilation	Generates the Python gRPC stubs (`pipeline_control_pb2.py`) in the client directory.
Service startup	Starts both services immediately.

4.3

Uninstallation

```
sudo ./uninstall.sh
```

This stops and disables the systemd services, removes the header symlinks, and uninstalls the gRPC Python packages.

5

Verifying the Services

After installation, both platform services run in the background. Verify their status:

```
systemctl status xrcomm-server.service
systemctl status xrcomm-grpc-ctrl.service
```

Both should report `active (running)`.

To inspect live logs:

```
journalctl -u xrcomm-server.service -f        # Platform Driver logs
journalctl -u xrcomm-grpc-ctrl.service -f      # Control Service logs
```

To restart the services after a configuration change:

```
sudo systemctl restart xrcomm-server.service
sudo systemctl restart xrcomm-grpc-ctrl.service
```

6

Controlling the Platform over gRPC

The remote control client is designed to run from the **Client Machine**.

6.1

Endpoint

The Control Service listens on `0.0.0.0:50051` by default. It starts automatically as a systemd service and does not need to be launched manually.

6.2

Client Setup

The Python gRPC client can be run from **any machine** (specifically the **Client Machine**) that has network access to the FlexCompute Server:

- **From the FlexCompute Server itself** (via loopback `127.0.0.1`) for internal verification during installation.
- **From a remote Client Machine** for standard operational use.

Install the Python dependencies once per machine:

```
cd XRComm_FCS_8T8R_platform/XRComm_platform_gRPC/client
pip3 install -r requirements.txt
```

The installer already generates the protobuf Python bindings on the FlexCompute Server. If you are setting up the client on a separate machine, copy the `client/` directory there and generate the stubs:

```
python3 -m grpc_tools.protoc -I. --python_out=. \
    --grpc_python_out=. pipeline_control.proto
```

6.3

Control Commands

Usage syntax:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 <command> [value]
```

Replace `<server-ip>` with the IP address of your FlexCompute Server (use `127.0.0.1` when running locally).

Command	What it does
`get-flags`	Show current delivery and recording mode flags.
`set-ip on|off`	Turn IQ-sample delivery to applications on or off.
`set-dsp on|off`	Turn full-packet delivery to applications on or off.
`set-logging on|off`	Start or stop NVMe recording.
`get-stats`	Show pipeline counters and platform telemetry.
`set-read-capture <ch> <start_ts> <duration>`	Set per-channel auto-read start offset and duration, in seconds.
`set-read-rate <ch> <hz>`	Set the per-channel receiver sample rate used to size auto-read output.
`get-read-config`	Show all 8 per-channel auto-read rows.
`get-read-status`	Show live NVMe auto-read progress by channel.
`list-ips`	List the applications currently connected to the platform.
`watch-status`	Stream live status once per second (Ctrl-C to stop).
`shutdown`	Gracefully shut the platform down.

6.4

8T8R Auto-Read Configuration

The 8T8R platform performs automatic NVMe read-back after a capture ends through `set-logging off`. Read-back is configured per channel in `XRComm_FCS_8T8R_platform/config/read_config.ini`. The installer exports this absolute path as `XRCOMM_READ_CONFIG` for both platform services.

Each channel has its own section:

```
[channel.0]
ch_no       = 0
start_ts    = 0
duration    = 1
sample_rate = 1966080000
```

The same schema repeats through `[channel.7]`. Do not copy the Wideband `[READBACK]` config into the 8T8R platform; the schemas are different.

Configure the values through gRPC before stopping logging:

```
C_PLAT="python3 xrcomm_grpc_client.py --host <server-ip>:50051"
$C_PLAT set-read-rate 0 1966080000
$C_PLAT set-read-capture 0 0.0 1.0
$C_PLAT get-read-config
```

End-to-end auto-read test:

```
$C_PLAT set-logging on
# Send or receive trigger-marked traffic for the active channels.
$C_PLAT set-logging off
$C_PLAT get-read-status
$C_PLAT watch-status 10
```

Auto-read logs only channels that captured data. Output is written under the Platform Driver working directory as `output_logs/ch<N>/event_<id>_<timestamp>/`, so the channel index and event timestamp are visible in the directory path.

7

Controlling the Playback over gRPC

The 8T8R variant includes the Universal Playback Engine (`XRComm_8T8R_playback/`). The gRPC server and client are designed to control multi-channel waveform streaming.

7.1

Separate Server Warning

> [!IMPORTANT] **Playback Hardware Separation:** The 8T8R Playback Engine (`xrcomm_playback`) streams raw IQ signal data at full line-rate (up to two 100 GbE ports simultaneously). To prevent host resource contention (such as CPU scheduling latency, memory bus saturation, or PCIe contention) with the receiver platform on the FlexCompute Server, **the Playback Engine must be executed on a separate physical server**. The Playback Server is cabled directly via high-speed fiber to the FlexCompute Server's receive NIC. Do not run the playback engine on the FlexCompute Server in production.

7.2

Verifying the Shipped Playback Binaries

The 8T8R Playback Engine is shipped with precompiled binaries. No local CMake build is required. Confirm that the playback engine and control server are present:

```
cd XRComm_8T8R_playback
ls -lh bin/xrcomm_playback bin/playback_grpc_server.py
```

7.3

Playback Server & Client Setup

The playback gRPC server binds to `[::]:50051` by default on the Playback Server. The Python gRPC stubs are precompiled and shipped with the package, so stub generation is not required. Starting this gRPC server is the only playback-side process customers need to launch manually; it drives the shipped `xrcomm_playback` binary.

Start the server manually from the Playback Server:

```
cd XRComm_8T8R_playback/bin
python3 playback_grpc_server.py
```

Set up client dependencies on the **Client Machine**:

```
cd XRComm_8T8R_playback/grpc/client
pip3 install -r requirements.txt
```

7.4

Playback Control Commands

Use `playback_client.py` on the **Client Machine** to configure and control the playback engine:

```
python3 playback_client.py --target <server-ip>:50051 <command> [args]
```

Command	Arguments	What it does
`get-config`	None	Read all 16 current `config.ini` values (read-only snapshot)
`get`	`core_list | source_mode | sample_rate | loop_count | trigger_burst_size`	Read one scalar configuration field
`set`	`field value`	Write one scalar configuration field (e.g. `set sample_rate 1966080000`)
`get-channel`	`<channel_id>` (0..7)	Read the filename configured for the specified channel
`set-channel`	`<channel_id> <filename>`	Set the waveform file for a channel (use `""` to disable the channel)
`get-txport`	`<port_id>` (0..1)	Read the DPDK transmit port ID for a port
`set-txport`	`<port_id> <tx_port_id>`	Set the DPDK transmit port ID for a port

Command	Arguments	What it does
`get-destmac`	`<port_id>` (0..1)	Read the destination MAC address for a port
`set-destmac`	`<port_id> <mac>`	Set the destination MAC address for a port (format `XX:XX:XX:XX:XX:XX`)
`start`	None	Launch playback using the current `config.ini` parameters
`stop`	None	Cleanly stop the running playback engine
`diagnostics`	None	Take a single snapshot of the 13 steady-state diagnostic metrics
`watch`	None	Stream diagnostics once per second (Ctrl-C to stop)

7.4.1 Active Channels & Parameters Configuration

- **Active Channels:** Any channel with a non-empty waveform filename is marked active. Each of the 8 channels can use a different waveform file; set an empty filename (`""`) to disable a channel. Querying `get-config` lists all active files and enabled channels.
- **Sample Rate:** Set globally using `set sample_rate <hz>`.
- **Start Triggers:** 8T8R playback supports independent per-channel trigger timing/offset behavior in the playback engine. The shipped gRPC client exposes waveform, port, MAC, start/stop, and diagnostics controls; do not interact with the playback binary console directly unless XRComm provides a delivery-specific trigger-offset interface.

8
Playback Operation

8.1
8T8R Platform Controls (Client Machine)

```
# Platform gRPC Client
C_PLAT="python3 xrcomm_grpc_client.py --host <server-ip>:50051"
$C_PLAT get-flags
$C_PLAT set-ip on
$C_PLAT set-logging on
```

8.2
8T8R Playback Controls (Client Machine)

```
# Playback gRPC Client
C_PLAY="python3 playback_client.py --target <playback-server-ip>:50051"
$C_PLAY get-config
$C_PLAY set core_list 6,7,8,9,10
$C_PLAY set sample_rate 1966080000
$C_PLAY set-channel 0 <waveform-file>
```



```
$C_PLAY set-channel 1 ""
$C_PLAY set-destmac 0 <dest-mac>
$C_PLAY start                          # Launch streaming
$C_PLAY watch                          # Monitor stream rates
$C_PLAY stop                           # Stop streaming
```

9 NVMe Logging, Auto-Read, and Optional Offline Retrieval

Start and stop NVMe logging captures via the platform gRPC client:

```
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging on   # Start logging
python3 xrcomm_grpc_client.py --host <server-ip>:50051 set-logging off  # Stop logging -> auto-read
```

The 8T8R platform records IQ samples directly to the NVMe disk via SPDK. Stopping logging through gRPC starts automatic read-back for active captured channels using `XRComm_FCS_8T8R_platform/config/read_config.ini`.

The offline NVMe reader is an optional tool for manual extraction after stopping the platform:

```
cd XRComm_NVMe_loader/scripts
sudo ./read_nvme.sh
```

Use `--first-gb=<N>` or `--last-mb=<N>` parameters on `xrcomm_nvme_reader` for partial extractions.

10 Appendix A: Environment Prerequisites

These configurations are **prerequisites** for the FlexCompute Server. They are typically set up once during initial server provisioning. If the platform fails to start or behaves unexpectedly, verify these settings first.

10.1 Dependency Version Check Table

Dependency	Required Version	Installation Command / Source
Ubuntu Linux	22.04 LTS or 24.04 LTS (64-bit)	Operating System standard install
DPDK	>= 23.11.1	`sudo apt install dpdk libdpdk-dev`; docs: https://core.dpdk.org/doc/
SPDK	>= 24.01	Build from source; docs: https://spdk.io/doc/
gRPC & Protobuf	v1.62.0 (C++ runtime)	`sudo apt install libgrpc++-dev libprotobuf-dev`
Python 3	v3.10+	Operating System standard install
grpcio & tools	v1.62.0 (Python runtime)	`pip3 install grpcio grpcio-tools`
CMake	v3.16+	`sudo apt install cmake`
GCC	v11+	`sudo apt install build-essential`

10.2

Hugepages

File-backed hugepages (2 MB size) are required for zero-copy memory pools:

```
sudo rm -rf /dev/hugepages/* /mnt/huge/*
sudo sysctl -w vm.nr_hugepages=0
echo 4096 | sudo tee /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages
# Mount the hugepage filesystem if not already mounted
sudo mkdir -p /dev/hugepages
sudo mount -t hugetlbfs nodev /dev/hugepages
```

10.3

Device Binding (DPDK / SPDK)

Before launching the server, identify the 8T8R NIC interfaces and NVMe storage drives, then bind them to the correct DPDK/SPDK user-space drivers.

- **Network Interface Discovery:** View available NIC PCI addresses, kernel interface names, and MAC addresses before binding:
  ```
  sudo dpdk-devbind.py --status
  ip -br link
  ethtool -P <interface-name>
  ```
  Select the NIC ports physically connected to the 8T8R RF/playback path, then use their MAC addresses when configuring receive/playback peers.
- **Network Interface (DPDK Binding):** Bind the selected transmit/receive NIC ports to the DPDK-compatible driver (`vfio-pci`) using their PCI addresses:
  ```
  sudo modprobe vfio-pci
  sudo dpdk-devbind.py --bind=vfio-pci <nic-pci-address> [<nic-pci-address> ...]
  ```
- **NVMe Drive (SPDK Binding):** Bind NVMe drives to SPDK using the SPDK setup script.

